Programming II

Inheritance and Polymorphism

Michele Pasqua, PhD

UNIVERSITÀ
di VERONA
Dipartimento
di INFORMATICA

A.Y. 2025/2026

Inheritance

Not inheritance

Subtypes

A type can be seen as the set of all values having something in common

Prime: integers ≥ 1 with only two divisors $\subseteq$ **Nat**: integers ≥ 0

If all values of a type belong to another type, the former is a subtype of the latter

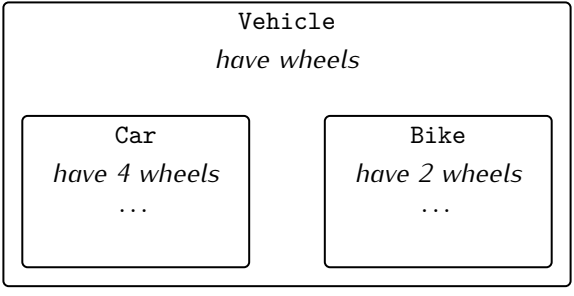
Classes are types, as they represent set of objects with common data/operations

- A class can be a subtype, or subclass, of another class

Subclass

The class `Car` is a subclass of the class `Vehicle`

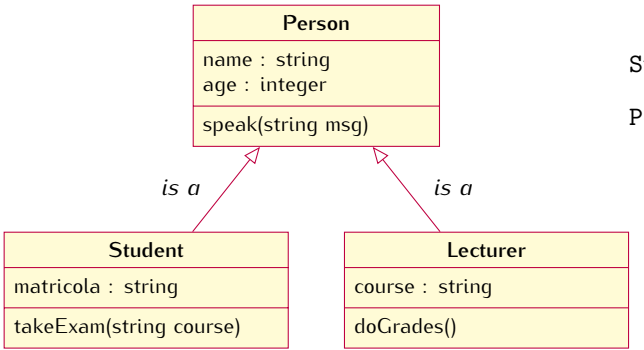
- Also the class `Bike` is a subclass of `Vehicle`
- But it is not a subclass of `Car`



Subclass (cont'd)

Every student is a person, but not a lecturer

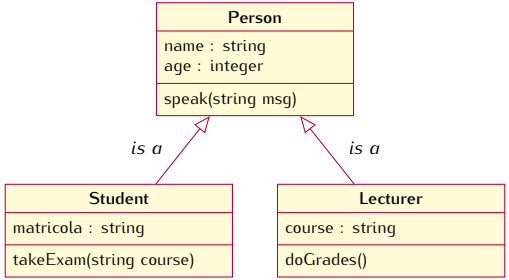
- `Student` and `Lecturer` objects have everything that a `Person` object has



`Student` is a subclass of `Person`

`Person` is a superclass of `Student`

Inheritance



A subclass inherits all fields and methods of its superclass

- A subclass is said to be derived from its superclass

Inheritance (cont'd)

A subclass can *// changes are available in the subclass only*

- Redefine, or override, some methods inherited from its superclass
- Add new fields and methods

```
public class Student extends Person { ... }
```

Java inheritance offers only one kind of derivation

- Class, field and method visibility cannot deviate from the superclass

Superclass

```
public class Vector {                                                    1
  private int[] vect;                                                    2
  public void putElement(int e) { ... } // append element                3
}                                                                        4
                                                                         5
public class OrderedVector extends Vector {                              6
  private Order order; // added field                                    7
  // overridden method                                                   8
  public void putElement(int e) {                                        9
    if (order == Order.ASC) { ... } // append and sort ascending         10
    else { ... } // append and sort descending                           11
  }                                                                      12
  public Order getOrder() { return order; } // added method             13
}                                                                        14
```

Can we reuse code from a superclass?

Superclass (cont'd)

With the keyword `super` we can refer to the parent class object

- So that we can call the parent class methods

```
public class Vector {                                                    1
  private int[] vect;                                                    2
  public void putElement(int e) { ... } // append element                3
}                                                                        4
                                                                         5
public class OrderedVector extends Vector {                              6
  ...                                                                    7
  public void putElement(int e) {                                        8
    super.putElement(e); // append is now here                           9
    if (order == Order.ASC) { ... } // just sort ascending here          10
    else { ... } // just sort descending here                            11
  }                                                                      12
}                                                                        13
```

Why inheritance?

Often, a class is just a modification of another class

- Inheritance minimizes the repetition of the same code

Localization of code

- Fixing a bug in a class automatically fixes it in the subclasses
- Adding a new feature to a class automatically adds it in the subclasses too
- Less chances of different (inconsistent) implementations of the same feature

Inheritance nomenclature

Parent class

- Class one above

Child class

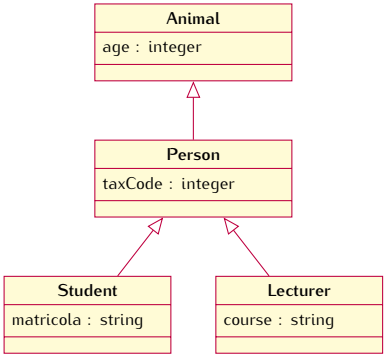
- Class one below

Superclass, or ancestor class, or base class

- Class one or more above

Subclass, or descendant class

- Class one or more below



Java does not have multiple inheritance

Method and field lookup

The JVM looks for method definitions in the superclass chain

- If a method is defined (or redefined) in the current class, it is called
- If not found, the method is searched in the parent class
- If not found, the method is searched in the parent of the parent class
- ... // *the search is guaranteed stop at some point*

⚠ Technically also fields can be redefined, but it is better to not do it

Explicit override

When redefining a method, always use the `@Override` annotation

- To make the code more understandable
- To avoid bugs (accidental overload instead of override)

If a `@Override` annotated method is not in a superclass, we get a compiler error

```
public class Vector {                                                    1
  public void putElement(int e) { ... }                                  2
}                                                                        3
                                                                         4
public class OrderedVector extends Vector {                              5
  @Override                                                              6
  public void putElment(int e) { ... } // compilation error              7
}                                                                        8
```

Subclass visibility

```
public class Person { // declared in package A                                  1
  String name;                                                                  2
}                                                                               3
                                                                                4
public class Student extends Person { // declared in package B                  5
  String matricola;                                                             6
  void printInfo() {                                                            7
    System.out.println(name + "␣" + matricola); // compilation error            8
  }                                             // name not accessible         9
}                                                                               10
```

When omitted, the default class member access is package-private

- It does not grant access to subclasses from other packages

Class member visibility

The `protected` access modifier grants access to subclasses from other packages

Class access modifiers

- `public`
- package-private (omitted)

Class member access modifiers

- `public`
- `protected`
- package-private (omitted)
- `private`

Access rules revisited

Access rules for class members (fields and methods)

	same package		other package	
	same class	other class	subclass	not subclass
`private` member in any class	✓	✗	✗	✗
(package) member any class	✓	✓	✗	✗
`protected` member in (package) class	✓	✓	✗	✗
`public` member in (package) class	✓	✓	✗	✗
`protected` member in `public` class	✓	✓	✓	✗
`public` member in `public` class	✓	✓	✓	✓

Class Hierarchy

The one ring

All classes in Java are descendants of the `Object` class

- Library class, user-defined class, wrapper class, enum, array

The compiler implicitly performs the derivation

```
class Car { ... }  ⟶  class Car extends Object { ... }
```

The `Object` contains some utility methods inherited by all classes

- `public String toString()` // *returns the object reference as a string*
- `public boolean equals(Object obj)` // *compares object references*
- `public int hasCode()` // *returns a unique id for the object*

The one ring (cont'd)

Often, the methods inherited from `Object` are not satisfactory

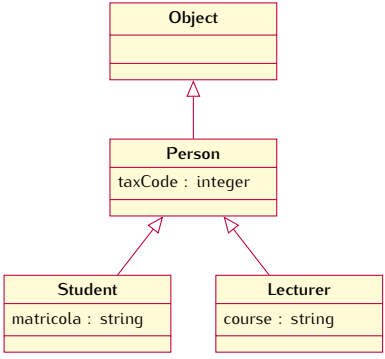
- They are typically redefined in subclasses

```
class Person {
  String name = "Sam";

  @Override
  public String toString() { return name; }

  void printPerson() {
    System.out.println(super.toString()); // prints 'Person@5ca881b5'
    System.out.println(this.toString());  // prints 'Sam'
  }
}
```

Origin class and superclasses

In `Student sam = new Student();`

- The origin class of the object `sam` is `Student`
- The superclasses of the object `sam` are `Person` and `Object`

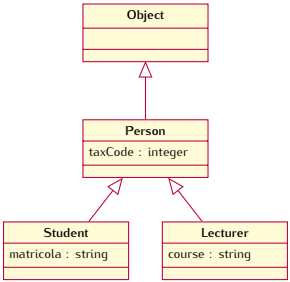


Origin class and superclasses (cont'd)

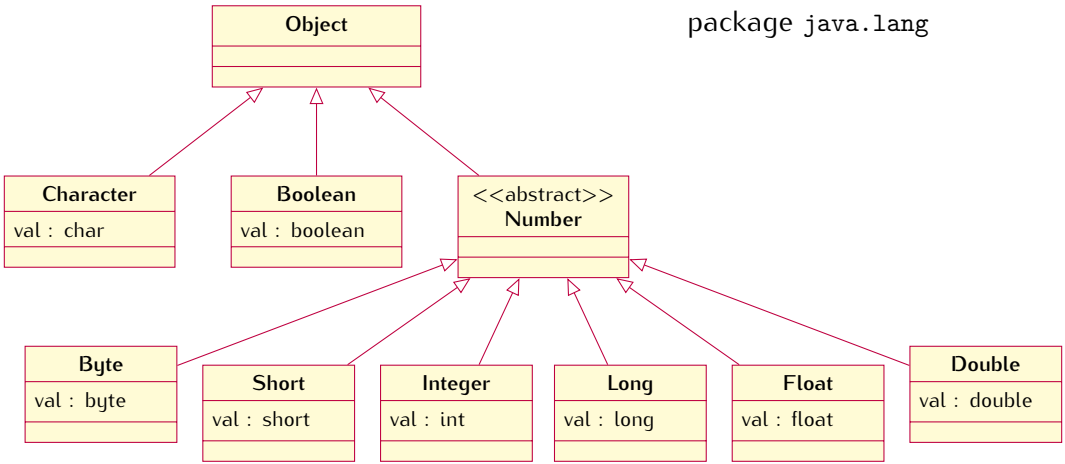
Java allows to check if a class is the origin class or a superclass of a given object

```
obj instanceof Clazz
```

```
Student sam = new Student();                1
                                            2
(sam instanceof Student)   // true          3
(sam instanceof Person)    // true          4
(sam instanceof Lecturer)  // false         5
(sam instanceof Object)    // always true   6
```



Wrapper class hierarchy



Object equality vs object identity

The `equals()` method is supposed to check object equality

- Its implementation in `Object` check reference equality (object identity)

```
class Person {
  String name;
  int age;
}

Person sam = new Person("Sam", 21);
Person alsoSam = new Person("Sam", 21);
System.out.println(sam.equals(alsoSam)); // the output is 'false'
```

Here `sam` and `alsoSam` are not the same object, but they are equal

Object equality vs object identity (cont'd)

Override the `equals()` method to get the expected result

- Two `Person` objects are equal when they have the same name and age

```
@Override
public boolean equals(Object obj) {
  if (obj == this) return true; // check object identity first

  if (obj == null) return false;
  if (!(obj instanceof Person)) return false; // then check type compliance
  Person other = (Person) obj; // explicit cast, see later

  // finally go inside the object to compare field by field
  if (this.age != other.age) return false;
  if (this.name == null)
    if (other != null) return false;
  else
    if (!this.name.equals(other.name)) return false;

  return true; // if all checks pass the objects are equal
}
```

Equality contract

An `equals()` method implementation should enforce some constraints

Equivalence relation axioms

- **Reflexivity** `x.equals(x) == true`
- **Symmetry** `x.equals(y) == y.equals(x)`
- **Transitivity** `x.equals(y) == true == y.equals(z)` implies `x.equals(x) == true`

Consistency

- Multiple executions of `x.equals(y)` must return the same result
- No object is equal to `null`, namely `x.equals(null) == false`

Inheritance and constructors

Since a subclass extends its parent class, we need (at least) two constructors

- The compiler implicitly calls the default constructor of the parent class
- Parent class constructor is executed before the child class constructor

Execution of constructors proceeds top-down in the inheritance hierarchy

- When a method of the child class is executed (constructor included), the superclass is completely initialized already

Constructors super power

With `super()` a constructor can refer to the parent constructors

```
class Person {                                                    1
  String name;                                                    2
  Person() { } // default constructor implicitly added            3
}                                                                 4
                                                                  5
class Student extends Person {                                    6
  int matricola;                                                  7
  Student() {  // default constructor implicitly added            8
    super();   // hidden call to the parent constructor           9
  }                                                               10
}                                                                 11
```

// Lines 3, 8, 9 and 10 will not actually appear in the code

Constructors super power (cont'd)

If a parameterized constructor is defined in the parent class

- No default constructor is inserted in the child class

```
class Person {                                                          1
  String name;                                                          2
  Person(String name) { this.name = name; } // parameterized constructor 3
}                                                                       4
                                                                        5
class Student extends Person {                                          6
  int matricola;                                                        7
  // we cannot compile this class                                       8
}                                                                       9
```

Either add a default constructor `Person() { }` in the parent class or add a constructor in the child class calling `super("Sam")` as first instruction

Super overloading

Since we may have many constructors, overloading allows `super()` to pick one

```
class Person {                                                          1
  String name;                                                          2
  Person() { } // default constructor                                   3
  Person(String name) { this.name = name; } // parameterized constructor 4
}                                                                       5
                                                                        6
class Student extends Person {                                          7
  int matricola;                                                        8
  Student(String name, int matricola) {                                 9
    super(name); // this must be the first instruction                  10
    this.matricola = matricola;  // the parent constructor is selected  11
  }                              // by signature match                 12
}                                                                       13
```

Final remark

One can deny the overriding of a method by using the `final` keyword

- Useful when the method must not change
- For instance, when the method provides basic service to other methods
- Trying to override a `final` method will result in a compiler error

Polymorphism

Same term different behaviors

Subtype polymorphism

A reference of type T can point to an object of type S if and only if

S is T or S is a subclass of T

```
Person person; // class Student extends class Person
person = new Person("Paul");       // ok -- same type
person = new Student("Sam", 123); // ok -- subtype
```

Substitution principle

If S is a subclass of T, then T objects can be replaced by S objects

- Known as Liskov Substitution Principle

```
Person person = new Person("Paul");
Student student = new Student("Sam", 123);

person = student;  // ok -- subtype substitution
student = person;  // error -- supertype substition
```

Substitution principle (cont'd)

The compiler performs method invocations on the basis of a reference's type

- The reference `ref` is of type `Person`
- We want to call `ref.getName()`
- Is the method `getName()` defined in the `Person` class?

If the reference's type does not provide the method we have a compiler error

- Subtyping guarantees that such a method do exist

Substitution principle rationale

The type of a reference establishes the methods one can call on an object

```
class Person {
  String name;
  Person(String name) {
    this.name = name;
  }
  String getName() {
    return name;
  }
}

Person person = new Student("Sam", 123); // ok
```

```
class Student extends Person {
  int matricola;
  Student(String name, int matricola) {
    super(name);
    this.matricola = matricola;
  }
  int getMatricola() { return matricola; }
}

Student student = new Person("Sam"); // error
          // substitution principle violation
```

`person.getName()` 

- Method found (inherited)

`student.getMatricola()` 

- Method not found

Dynamic binding

What about overridden methods?

```
class Person extends Object {                                          1
  String name;                                                         2
  Person(String name) { this.name = name; }                            3
  @Override   // redefine the toString() method of the class Object    4
  public String toString() { return name; }                            5
}                                                                      6
                                                                       7
Object obj = new Person("Sam");                                        8
```

Which method is selected when `obj.toString()` is called?

- Both classes `Object` and `Person` provide the method `toString()`

Dynamic binding (cont'd)

How methods are resolved by the JVM

- The JVM retrieves the origin class of the target object
- If the class contains the method to call it is executed
- Otherwise, the parent class is considered and the previous step repeated
- The procedure is guaranteed to eventually stop

The compiler checks that the reference's class hierarchy contains the method

Type conversion

Java is strongly typed, each variable must have a type at compile time

```
float f;
f = 4.7f; // ok -- float value to float variable
f = true; // error -- boolean value to float variable

Car c;
c = new Car();     // ok -- Car object to Car variable
c = new String(); // error -- String object to Car variable
```

Type conversion (cont'd)

Type conversion: sometimes it is possible to change the type of an expression

- Explicitly or implicitly (by the compiler)

```
int i;
float f;

f = i; // implicit cast from int to float

f = (float) 42; // explicit cast from int to float
```

Type conversion is also referred as casting

Primitive type casting

Java performs an implicit cast from “smaller” types to “bigger” types

- But not vice versa *// we need to force casting*

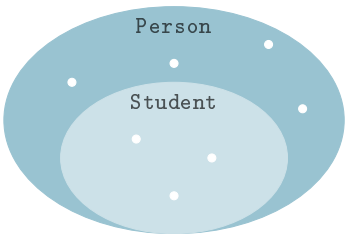
```
int i = 3;
float f = 2.4f;

f = i;                  // ok -- legit cast (float bigger than int)
System.out.println(f); // the output will be '3.0'

i = f; // error -- not legit cast (int smaller than float)

i = (int) f;            // ok -- forced cast (with information loss)
System.out.println(i); // the output will be '2'
```

Class upcasting

Upcast: from a more specific type (subtype) to a more general type (supertype)

- Upcasting is implicitly performed by the compiler
- The substitution principle guarantees type-safety

```
// Student extends Person                          1
Person person = new Person("Paul");                2
Student student = new Student("Sam", 123);         3
                                                   4
person = student; // implicit upcast               5
```



$\forall \texttt{obj} \in \texttt{Student} \,.\, \texttt{obj} \in \texttt{Person}$

Class upcasting rationale

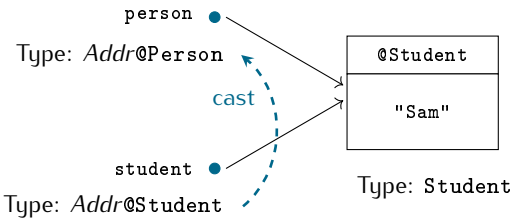
We can act in the same way on objects from different classes

- Provided that they inherit from a common class

```
Person[] courseParticipants = {
  new Lecturer("Paul", "Programming II"),
  new Student("Sam", 123),
  new Student("Gary", 456)
};

for (Person person : courseParticipants)
  System.out.println(person.toString());
```

It is all about references

Reference type and object type are distinct concepts




```
Person person = null;
Student student = new Student("Sam", 123);

person = student;
```

Cast only affects the reference, the pointed object (value) does not change

- Only with primitive types casting involves a value conversion

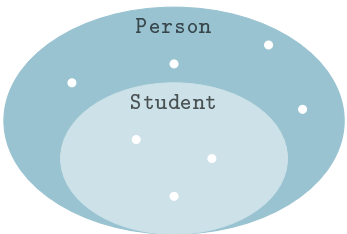
Class downcasting

Downcast: from a more general type (supertype) to a more specific type (subtype)

- Downcasting must be explicitly forced by the programmer
- Not guaranteed to be type-safe

// is the below example safe?

```
// Student extends Person                          1
Person person = new Person("Paul");                2
Student student = new Student("Sam", 123);         3
                                                   4
student = (Student) person; // explicit            5
                           // downcast             6
```



$\exists \texttt{obj} \in \texttt{Person} \,.\, \texttt{obj} \notin \texttt{Student}$

Class downcasting rationale

To access a member defined in a class you need a reference of that class type

- Or any subclass

```
Person person = courseParticipants[1];                                    1
System.out.println(person.getMatricola());  // compiler error             2
            // the class Person does not know the getMatricola() method   3
                                                                          4
Student student = (Student) person;                                       5
System.out.println(student.getMatricola()); // ok for the compiler        6
                   // the class Student knows the getMatricola() method   7
```

...provided that `courseParticipants[1]` is a `Student` object

Pay attention to downcasts

The compiler trusts any downcast (within the inheritance hierarchy)

- But the JVM checks at runtime reference consistency
- Incorrect downcasts yield an exception

```
// Student extends Person                                                    1
Person person = new Person("Paul");                                          2
Student student = new Student("Sam", 123);                                   3
                                                                             4
student = (Student) person; // runtime error -- ClassCastException           5
```

Compile-time and runtime errors

```
Person person = new Student("Sam", 123);
```

`person.getMatricola()`	✗	compile-time error
`((Student) person).getMatricola()`	✓	no errors
`((String) person).getMatricola()`	✗	compile-time error
`((Lecturer) person).getCourse()`	✗	runtime error

Downcast safety

To avoid runtime errors, use `ref instanceof Clazz`

- To checks if the object referenced by `ref` can be casted to `Clazz`
- To checks if it belongs to `Clazz` or any of its subclasses

```
Person person = courseParticipants[1];

if (person instanceof Student) {
  Student student = (Student) person; // no ClassCastException
  System.out.println(student.getMatricola());
}
```

Q + A